Assignments-5

Aim:

To implement **Breadth-First Search (BFS)** to solve a planning problem such as **Robot Path Planning**.

Problem Statement

Consider a robot placed in a grid environment. The robot must move from a **Start position (S)** to a **Goal position (G)** while avoiding obstacles. Implement BFS to find the **shortest path** from the start position to the goal.

Objectives

- Understand BFS as an **uninformed search technique**.
- Represent a planning problem using **states and actions**.
- Find the shortest path between two states.
- Display the sequence of actions required to reach the goal.

Example

Grid:

```
S  0  0  X  0
0  X  0  X  0
0  X  0  0  0
0  0  0  X  G
```

Where:

- `S` = Start position
- `G` = Goal position
- `0` = Free cell
- `X` = Obstacle

State Representation

A state is represented as:

```
(row, column)
```

For example:

```
Start = (0, 0)
Goal  = (3, 4)
```

Actions

The robot can move:

```
UP
DOWN
LEFT
RIGHT
```

Algorithm

1. Initialize a queue with the start state.
2. Mark the start state as visited.
3. Remove the first state from the queue.
4. Generate all valid neighboring states.
5. If a neighbor is the goal, return the path.
6. Otherwise, add the unvisited neighbor to the queue.
7. Repeat until the goal is reached or the queue becomes empty.

Python Program:

```
from collections import deque

# Grid
grid = [
    ['S', '0', '0', 'X', '0'],
    ['0', 'X', '0', 'X', '0'],
    ['0', 'X', '0', '0', '0'],
    ['0', '0', '0', 'X', 'G']
]

rows = len(grid)
cols = len(grid[0])

# Find start and goal
for i in range(rows):
    for j in range(cols):
        if grid[i][j] == 'S':
            start = (i, j)
        elif grid[i][j] == 'G':
            goal = (i, j)

# Possible movements
moves = [
    (-1, 0, "UP"),
    (1, 0, "DOWN"),
    (0, -1, "LEFT"),
    (0, 1, "RIGHT")
]

def bfs(start, goal):
    queue = deque([(start, [])])
```

```
    visited = {start}

    while queue:
        current, path = queue.popleft()

        if current == goal:
            return path

        for dr, dc, action in moves:
            nr = current[0] + dr
            nc = current[1] + dc

            if (0 <= nr < rows and
                0 <= nc < cols and
                grid[nr][nc] != 'X' and
                (nr, nc) not in visited):

                visited.add((nr, nc))
                queue.append(
                    ((nr, nc), path + [action])
                )

    return None

# Find shortest path
path = bfs(start, goal)

if path:
    print("Shortest Path:")
    print(" -> ".join(path))
    print("Number of steps:", len(path))
else:
    print("No path found")
```

Expected Output

```
Shortest Path:
DOWN -> DOWN -> DOWN -> RIGHT -> RIGHT -> UP -> UP -> RIGHT -> DOWN

Number of steps: 7
```

Conclusion:

Thus, the **BFS algorithm was successfully implemented to solve a robot path-planning problem and find the shortest path from the initial state to the goal state**.

Output :

```
                grid[nr][nc] != 'X' and
                (nr, nc) not in visited):

                visited.add((nr, nc))
                queue.append(
                    ((nr, nc), path + [action])
                )

    return None

# Find shortest path
path = bfs(start, goal)

if path:
    print("Shortest Path:")
    print(" -> ".join(path))
    print("Number of steps:", len(path))
else:
    print("No path found")
```

```
Shortest Path:
RIGHT -> RIGHT -> DOWN -> DOWN -> RIGHT -> RIGHT -> DOWN
Number of steps: 7
```

In [ ]:

Activate Windows